

Implement the Merge Sort algorithm in Python. Given an unsorted array, apply the Merge Sort to sort it in ascending order. Provide the pseudocode and Python code for the Merge Sort algorithm.

Pseudo Code

MERGESORT(A, p, r)

1. If $p < r$
2. $q = \text{floor}((p + r) / 2)$
3. MERGESORT(A, p, q)
4. MERGESORT(A, q + 1, r)
5. MERGE(A, p, q, r)

MERGE(A, p, q, r)

1. Let $n1 = q - p + 1$
2. Let $n2 = r - q$
3. Create arrays $L[1..n1+1]$ and $R[1..n2+1]$
4. For $i = 1$ to $n1$
5. $L[i] = A[p + i - 1]$
6. For $j = 1$ to $n2$
7. $R[j] = A[q + j]$
8. $L[n1 + 1] = \infty$
9. $R[n2 + 1] = \infty$
10. $i = 1, j = 1$
11. For $k = p$ to r
12. If $L[i] \leq R[j]$
13. $A[k] = L[i]$

14. $i = i + 1$
15. Else
16. $A[k] = R[j]$
17. $j = j + 1$

Answer

```
def merge_sort(a, p, r):  
    if p < r:  
        q = (p + r) // 2 # Correct floor division using integer division  
        merge_sort(a, p, q)  
        merge_sort(a, q + 1, r)  
        merge(a, p, q, r)  
  
def merge(a, p, q, r):  
    n1 = q - p + 1  
    n2 = r - q  
  
    L = [0] * (n1 + 1) # +1 to store the sentinel value  
    R = [0] * (n2 + 1) # +1 to store the sentinel value  
  
    # Copy the data into L[] and R[]  
    for i in range(n1):  
        L[i] = a[p + i]  
    for j in range(n2):
```

```
R[j] = a[q + 1 + j]
```

```
# Set the sentinel values to infinity
```

```
L[n1] = float('inf')
```

```
R[n2] = float('inf')
```

```
i = 0
```

```
j = 0
```

```
# Merge the arrays back into a[]
```

```
for k in range(p, r + 1):
```

```
    if L[i] <= R[j]:
```

```
        a[k] = L[i]
```

```
        i += 1
```

```
    else:
```

```
        a[k] = R[j]
```

```
        j += 1
```

```
# Example usage
```

```
array = [38, 27, 43, 3, 9, 82, 10]
```

```
merge_sort(array, 0, len(array) - 1)
```

```
print("Sorted array:", array)
```